

# CaDiCaL 2.2: Proofs, Algorithms, Implied Literals, and Robust Scheduling

Florian Pollitt<sup>✉</sup><sup>1</sup>, Mathias Fleury<sup>1</sup>,  
Katalin Fazekas<sup>3</sup>, Nils Froleyks<sup>2</sup>, and Armin Biere<sup>1</sup>

<sup>1</sup> University of Freiburg, Freiburg, Germany

<sup>2</sup> Johannes Kepler University Linz, Linz, Austria

<sup>3</sup> TU Wien, Vienna, Austria

**Abstract.** The last SAT Competition 2024 was dominated by Kissat, the successor of CaDiCaL, due to improving several core solving techniques. The bare-metal approach of Kissat makes achieving these improvements easier to implement, as it does not need to support the rich feature set of an incremental SAT solver, such as CaDiCaL. However, many applications rely on these features. This system description reports on how we successfully adapted Kissat’s award-winning techniques to CaDiCaL, including clausal congruence closure, clausal equivalence sweeping, and bounded variable addition. The main challenge was to support efficient linear proof production with hints. We further extended the API to extract implied literals under assumptions and applied advanced deterministic scheduling of inprocessing based on the ticks metric for approximating cache line accesses. Experiments confirm the benefits of these efforts.

## 1 Introduction

This system description presents the substantial number of additions made available in the latest version 2.2 of our widely used SAT solver CADICAL. They improve performance considerably without compromise on the use model. We also added a feature of allowing users to extract implied literals under assumptions.

Besides maintaining its rich incremental API, the real challenge was to continue to support the unique feature of CADICAL of being able to produce linear resolution proofs with hints in form of antecedents. These proofs can readily be used for interpolation, can be checked much faster than proofs produced by other state-of-the-art solvers, and take almost no overhead to produce.

The CADICAL 2.2 extensions are motivated by the success of our bare-metal SAT solver KISSAT in the SAT Competition 2024, where it took first place in all categories of the main track. We attribute the success of KISSAT to the addition of new inprocessing algorithms, which includes a novel integrated inprocessing version of bounded variable addition, lucky phases (actually ported from CADICAL to KISSAT but then extended and now ported back to CADICAL), improved vivification and clausal sweeping, fast bounded variable elimination during preprocessing, and, last but not least, clausal congruence closure.

**Table 1.** Contributions and Feature Comparison with selected SAT solvers

| Solver             | Proofs                     | Algorithms        |              |         | API                   | Robustness |
|--------------------|----------------------------|-------------------|--------------|---------|-----------------------|------------|
|                    | Format                     | Congruence[13]    | Sweeping[15] | BVA[42] | Implied[52]           | Scheduling |
| GLUCOSE [2]        | LRUP[20, 27]               | —                 | —            | —       | ✓                     | —          |
| CRYPTOMINISAT [49] | FRAT[4]                    | XORs[50]          | —            | ✓       | —                     | conflicts  |
| LINGELING [7]      | DRUP[36]                   | simpleprobing[13] | BCE[33]      | —       | ✓                     | conflicts  |
| KISSAT [12]        | DRAT[51]                   | ✓                 | ✓            | ✓       | —                     | ticks      |
| CADICAL 2.0 [11]   | LRUP[20, 45]<br>LIDRUP[27] | —                 | —            | —       | IPASIR-UP<br>[25, 26] | conflicts  |
| CADICAL 2.2 [*]    | LRAT[20, *]<br>LIDRUPE[*]  | ✓[*]              | ✓[*]         | ✓[*]    | ✓[*]                  | ticks[*]   |

[\*] this paper

Furthermore, KISSAT implements advanced scheduling of interleaving inprocessing algorithms as well as interleaving *stable* and *unstable* mode during search (focusing on *satisfiable* and *unsatisfiable* instances respectively [44]). Scheduling in KISSAT relies on the “ticks” metric, which deterministically and precisely approximates run-time, while CADICAL 2.0 uses the more fluctuating metrics of conflicts and propagations (varying greatly between stable and unstable mode).

We succeeded to lift all these distinguishing techniques available in KISSAT to CADICAL and extended them to produce linear resolution (LRAT) proofs [21]. We describe a new API contract for incremental SAT solving which allows us to produce and check proofs for incremental solving, while introducing extension variables, as necessary for bounded variable addition. We are not aware of any other incremental proof-producing SAT solver that goes beyond resolution.

Table 1 lists related SAT solvers, GLUCOSE [2], CRYPTOMINISAT [49] and LINGELING [7], with similar incremental features. Below are KISSAT [12] and the previous version CADICAL 2.0 [11]. They yield the new version CADICAL 2.2. The table focuses on the new features, split into four categories, with the most challenging feature **LRAT proofs** first, second the major part on new inprocessing **algorithms**, third the **API** extension “implied”, and last scheduling for improving **robustness** of solving. The paper structure follows this categorization.

We have lifted several additional techniques proven to be useful from KISSAT to CADICAL 2.2. These include an improved form of vivification, lucky-phases with look-ahead, a fast dedicated variant of bounded variable elimination during preprocessing and semantic definition mining for variable elimination [29]. However, porting the KISSAT code was straight-forward because new features linear proof generation with hints is easy. Therefore we refer to the KISSAT system description [12] and [29] for more details on these.

## 2 Proofs

Certifying SAT solvers through proofs has a long history [32, 34]. The most common proof format is DRAT [51], which dates back to 2013, and has been the standard for the SAT Competition until 2023. Proof checking of DRAT

**Table 2.** Comparison of different proof formats

| <i>id</i> | DIMACS [41] | DRUP [36] | LRUP [20]    | LIDRUP [27] (without i-lines) |
|-----------|-------------|-----------|--------------|-------------------------------|
|           | p cnf 2 4   | -1 0      | 5 -1 0 4 3 0 | q 0                           |
| 1         | 1 2 0       | d -1 2 0  | 5 d 3 4 0    | 1 5 -1 0 4 3 0                |
| 2         | 1 -2 0      | d -1 -2 0 | 6 2 0 5 1 0  | d 3 4 0                       |
| 3         | -1 2 0      | 2 0       | 7 0 5 6 2 0  | 1 6 2 0 5 1 0                 |
| 4         | -1 -2 0     | 0         |              | 1 7 0 5 6 2 0                 |
|           |             |           |              | s UNSATISFIABLE               |
|           |             |           |              | u 0 7 0                       |

proofs, however is very expensive, even compared to solving. There were efforts of addressing this issue. On one side, more efficient and trustworthy checking flows were developed, with intermediate proof formats LRAT [20] and GRAT [40]. On the other side, attempts were made to replace DRAT with a more verbose and easier to check format, giving us FRAT [3, 4].

Going back in time, DRUP [36], the predecessor of DRAT, is based on resolution. DRAT is an extension which also supports redundant asymmetric tautologies, a stronger form of reasoning, no longer expressible by resolution only. Meanwhile there are solving techniques which require even stronger proof formats, such as DPR [35], an extension to DRAT, and a different proof format altogether based on pseudo-boolean proofs VeriPB [19]. In 2023, CADICAL was extended with full LRAT proof production capabilities or more precisely the LRUP subset [45].

In the world of incremental SAT solving, the first proof formats have been devised only recently, namely IDRUP [28] and LIDRUP [27]. They are based on DRUP and LRUP respectively, as the soundness argument of unconditional RAT-based clause addition breaks when adding clauses incrementally [24].

Table 2 illustrates some of the proof formats that are supported by CADICAL. The first column shows a small formula defined in the DIMACS format [41], and the other columns present the differences between the DRUP [36], LRUP [20], and (truncated) LIDRUP [27] refutations. While DRUP proofs explicitly list each learned and deleted clause of the solver, LRUP refers to clauses by an identifier (*id*) and provides additional hints on how learned clauses are derived. LIDRUP extends LRUP (as IDRUP extends DRUP) by including the interactions of the user during the solving process (such as formulating a query or extracting cores).

The newly introduced formula simplification techniques and solver features BVA (Sect. 5) and extracting implied literals (Sect. 6) required us to extend the proof producing abilities of CADICAL. How we extend (L)IDRUP to address these new techniques is outlined in the respective sections below.

### 3 Clausal Congruence Closure

One key new feature of KISSAT is clausal congruence closure [14]. It detects AND, XOR, and ITE (*if-then-else*) gates which are encoded in the CNF formula. When two gates of the same type have the same inputs, the output variables are semantically identical and can be merged, i.e., all occurrences of one can be

rewritten to the other. This simplifies the extracted gates, potentially leading to more equivalences or even units. The congruence closure algorithm exhaustively detects and merges equivalent variables, learns the corresponding clauses and simplifies the gates through rewriting and unit propagation. While focusing on gates, the CNF is left unsimplified until termination. In contrast, the XOR recognition from CRYPTOMINISAT [49] recognizes only XOR gates [47, 48] (*cf.* Tbl. 1).

The main challenge is to create LRAT proofs for the iterative merging process. For DRAT, the pattern of the merged gates is enough to establish the equivalence in the proof. That way, after gate extraction, the algorithm does not need to concern itself with actual clauses. For LRAT however, it is essential to keep track of gate clauses and the extent to which they have been simplified. This is highly non-trivial as simplification of gates is performed in parallel to the formula.

To synchronize their simplification we keep track of clauses corresponding to each gate and simplify them once the gate is merged and they are used for the proof. Simplified clauses are proof-only and kept separately. That way, solver behavior does not change between proof production and no proofs.

Eagerly rewriting *all* clauses after merging two literals is time-consuming. Therefore we tried a lazy approach instead, which attempted to determine if rewriting was needed during proof production. For instance, to resolve  $a \vee b$ ,  $\neg b \vee c$ , and  $\neg c' \vee d$  modulo the equivalences  $b = b'$  and  $c = c'$ , rewriting  $b$  is not needed but rewriting  $c$  is. This however turned out to be too complex. We now rewrite merged literals in clauses *only* on-the-fly just before using them.

Algorithmically, proving equivalence for XOR gates is interesting. Consider merging  $x = a \oplus b \oplus c$  with  $y = a \oplus b \oplus c$ . We want to derive the equivalence, i.e., the two clauses  $\neg x \vee y$  and  $x \vee \neg y$ . First, we rewrite the clauses to guarantee that they always have the same shape (variable suffix). Second, we pair up and resolve two corresponding clauses, one from each gate (e.g.,  $x \vee a \vee b \vee c$  and  $\neg y \vee a \vee b \vee \neg c$ ). This leaves us with half as many clauses (e.g., without  $c$ ). We iterate this process until we derive the two equivalence clauses. All intermediate clauses are added to the DRAT and LRAT proofs. To generate LRAT, we use the clause index (here a bitfield of the literal signs) to form the next resolution step.

Finally, supporting AND and especially ITE gates is way harder for LRAT with 6 new ITE rewriting rules compared to DRAT (one having 10 different cases for LRAT proofs). This is due to many special cases after simplification. For instance, an ITE gate  $x = (c ? t : e)$  has 4 definition clauses, two for  $c \rightarrow (x = t)$  and two for  $\neg c \rightarrow (x = e)$ . When  $x$  and  $t$  are the same, we only get the two clauses for  $e$ ; when  $c$  and  $t$  are the same, only three, leading to different proofs.

## 4 Clausal Equivalence Sweeping

Clausal equivalence sweeping [15] is another approach for finding equivalences within the formula. It mirrors hybrid SAT sweeping algorithms for the equivalence checking problem of circuits but works directly on the CNF. It employs a second SAT solver, which can focus on a subset of the original formula. This is the biggest

advantage compared to previous attempts at adapting circuit-level SAT sweeping, such as making use of blocked clause decomposition (BCE [5, 33], *cf.* Tbl. 1).

Following KISSAT we use KITTEN [29] as lightweight incremental subsolver and ported clausal equivalence sweeping [15] accordingly. During the integration of KITTEN in CADICAL, we fixed minor issues, which however cannot occur in KISSAT due to more restrictive usage. The integration into CADICAL also required adjustments to the API of the KITTEN solver and one algorithmic change to KITTEN’s version of “analyze-final”. This is due to different representations of clauses within KISSAT and CADICAL and, of course, because of LRAT proofs.

Each proof-step tried in KITTEN might yield a proof-step in the main solver. Because of that, KITTEN keeps resolution antecedents when creating new clauses. Thus, to obtain LRAT proofs in CADICAL, we map KITTEN antecedents to CADICAL clauses. The original algorithm in KISSAT simplifies clauses while giving them to KITTEN, i.e., replacing derived equivalences and removing fixed values. Tracking such simplification implicitly is complex and costly (*cf.* Sect. 3). Instead, we give fixed variables to KITTEN as unit clauses and explicitly apply equivalences in CADICAL between rounds of the algorithm.

## 5 Bounded Variable Addition

Bounded Variable Addition (BVA), originally devised in 2012 [42], uses extended resolution (or RAT reasoning) to reencode parts of the formula. It had a big resurgence in the SAT Competition 2023 [6]. The winner SBVA-CADICAL sc2023 [31] used a BVA preprocessor in combination with CADICAL 1.5.3. After this, we added BVA to KISSAT for the SAT Competition 2024 [12] as an inprocessing step. Porting this inprocessing version to CADICAL poses a unique challenge.

As discussed above, unconditional RAT addition is not compatible with incremental reasoning [24]. It is compatible however, if we restrict RAT addition to *latent* witness variables, i.e., internal *extension* variables which do not occur in input nor in eliminated clauses. Accordingly we extend LIDRUP to LIDRUPE and ensure correct handling of variables with a new API contract: For each new external variable the user has to ask the solver for a new variable index. Thus the solver can maintain a consecutive range of variables where user and (latent) solver variables are interleaved arbitrarily. This required various changes to API contracts and tooling, especially to our testing infrastructure with MOBICAL [11] and to proof checkers (LRAT-TRIM [45], LIDRUP-CHECK [27], IDRUP-CHECK [27]).

Consider the resulting proofs in different formats in Tbl. 3. We have four binary input clauses in DIMACS format to which we can apply BVA. To replace them we want to introduce clauses with *id*’s 5, 6, 8 and 9. They resolve on the newly introduced variable 5 to get back the original four input clauses.

That way, we essentially performed a backward variable elimination step on variable 5. In the proof there are three steps that are not plain resolution steps. The first two derived clauses (*id*’s 5, 6) are *pure* on the new variable 5, meaning that 5 is not contained negatively in the formula. This pure RAT step is denoted by empty antecedents in the LRAT proof. The third derived clause with *id* 7 is a

**Table 3.** Small example with a single BVA simplification

| <i>id</i> | DIMACS [41] | DRAT [51]    | LRAT [20]            | LIDRUPE [*] (without i-lines) |
|-----------|-------------|--------------|----------------------|-------------------------------|
|           | p cnf 2 4   | 1 5 0        | 5 1 5 0 0            | q 0                           |
| 1         | 1 2 0       | 3 5 0        | 6 3 5 0 0            | 1 5 1 5 0 0                   |
| 2         | 2 3 0       | -1 -3 -5 0   | 7 -1 -3 -5 0 -5 -6 0 | 1 6 3 5 0 0                   |
| 3         | 3 4 0       | 2 -5 0       | 8 2 -5 0 1 2 7 0     | 1 7 -1 -3 -5 0 -5 -6 0        |
| 4         | 4 1 0       | 4 -5 0       | 9 4 -5 0 3 4 7 0     | 1 8 2 -5 0 1 2 7 0            |
|           |             | d -1 -3 -5 0 | 9 d 7 1 2 3 4 0      | 1 9 4 -5 0 3 4 7 0            |
|           |             | d 1 2 0      |                      | d 7 1 2 3 4 0                 |
|           |             | d 2 3 0      |                      | s SATISFIABLE                 |
|           |             | d 3 4 0      |                      | m 1 2 3 4 0                   |
|           |             | d 4 1 0      |                      |                               |

[\*] this paper

helper clause and is *blocked* on the new variable (literal -5), meaning that all possible resolvents on variable 5 are tautologies. This blocked RAT step and is denoted by negative *id*'s in the LRAT proof. The remaining extension clauses (*id*'s 8, 9) are derived by resolution before deleting unnecessary clauses.

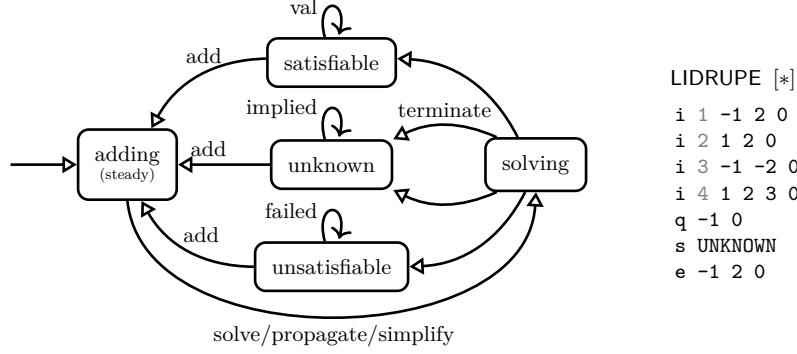
Our extension LIDRUPE copies the syntax from LRAT but needs to check that the API contract is kept, i.e. that latent variables introduced in lemma clauses (1-lines) never occur in input clauses (i-lines) nor queries (q-lines). Proof-checking also requires that the use of latent variables as witness literals for RAT steps are not passive (do not occur in weakened, i.e., not yet restored, clauses) [23,27].

## 6 Implied Literals

In many use cases of SAT solvers, e.g., MaxSAT preprocessing, SMT preprocessing, or backbone extraction [17], it is important to compute “immediate” consequences of a set of literals with respect to a formula, rather than searching for a complete model. To this extent incremental SAT solvers like MINISAT [22], GLUCOSE [2], or the PYSAT interface [37,38], typically provide users with a way to run limited searches (i.e., only *propagate*) on a given set of assumptions. As of yet, CADICAL supported this only through its IPASIR-UP interface [25], which however is a laborious way and has its own challenges in combination with other features (e.g., preprocessing and proof production). Thus, we included now a dedicated **propagate** function in CADICAL 2.2's external interface.

Fig. 1 shows the adapted main internal state graph of CADICAL 2.2 with the new propagation functionality. After initialization and clause additions, users can assume literals using the **assume** function [22]. Then, they can trigger the (potentially limited) search of the solver by calling **solve**, **propagate**, or **simplify**. Each query returns either **SATISFIABLE**, **UNSATISFIABLE**, or **UNKNOWN**.

Users can query the model or the core through **val** resp. **failed** functions. For an **UNKNOWN** result implied literals can be obtained via the **implied** function (as in LINGELING [7]). Note, that this “implied” set of literals is in general (in contrast to backbones [17]) only a subset of all logically entailed literals, i.e., the solver uses only unit propagation and ignores eliminated variables and clauses.



**Fig. 1.** Simplified internal state-transition-graph of a SAT solver (on the left) and a proof produced by a call to **propagate** followed by an **implied** call (on the right).

We extended the LIDRUP proof format accordingly: Instead of restricted tracing to the new propagation calls, we standardize handling of queries returning **UNKNOWN**, which also applies to preprocessing calls and asynchronously terminated search calls. As Fig. 1 illustrates, when neither a model nor a contradiction is found by the solver, LIDRUP requires an “**s UNKNOWN**” line and propagated literals can be included with the prefix “**e**” (short for *entailed* – as “**i**” is taken).

## 7 Robust Scheduling

Tuning SAT solvers is non-trivial [8]. For instance, CADICAL (and KISSAT) have many different search heuristics, including two different search modes, alternating *stable* and *unstable* search after the ideas of Chanseok Oh [44]. Additionally, they employ a wide range of pre- and inprocessing techniques [18]. Usefulness and run-time cost of these techniques vary wildly between benchmarks and are further hard to predict. To control and ultimately improve robustness, i.e., reduce solving time variance, we want a precise deterministic metric to restrict the time spent on different techniques and in different search modes. Classical metrics in SAT solving are based on the number of conflicts encountered or the number of propagations (number of unit-propagations of assigned variables).

These metrics do not correlate well with actual time spent. For instance during unstable mode with very frequent restarts the propagation rate (per second) can dramatically decrease compared to the stable mode with rare restarts, and accordingly for the conflict rate. Different clause variable ratios, the total size of the benchmark or different ranges of LBD levels [1] have similar influences.

Like many other solvers (*cf.* Tbl. 1), our previous version CADICAL 2.0 used conflicts to schedule different modes and inprocessing techniques. It further relied on counting propagations to limit the effort spent within individual inprocessing techniques. As argued above (and shown in our experiments) this leads to

substantial variations in time allocated to different techniques and for instance makes the split between stable and unstable mode during search less balanced.

To counter this effect, KISSAT introduced *ticks* [16], as an approximation to the number of cache line accesses, focusing on the hotspot of execution (usually the unit-propagation loop). Scheduling stable and unstable mode based on ticks balances the ratio of time spent in the two modes, bringing it closer to 50%.

The ticks metric can also be used to limit the cost of inprocessing techniques with respect to solving time. It was also effective in making parallel SAT solvers deterministic in context of the DPS framework [43]. We further argue that counting cache line accesses instead of the memory accesses (Knuth’s “mems” [39]) is more accurate. For instance during unit-propagation of binary clauses the other literal is stored in the watch list. Data of consecutively propagated binary clauses resides in the same cache line, and hence can be accessed at no cost, in contrast to dereferencing large clauses, distributed randomly in memory.

The new version of CADICAL 2.2 computes and makes use of ticks during search and most inprocessing techniques. We first added ticks counting to obvious hotspots, then determined outliers and iteratively adapted ticks computation through profiling. As a side effect, we also achieved a better understanding of algorithm costs, e.g., scheduling in vivification turned out to be more expensive than expected and had to be limited for large benchmarks. Ticks are also used to ensure even distribution between stable and unstable modes: the first (unstable) mode phase is limited by conflicts, ticks spent are measured and scaled to limit the time spent in subsequent stable and unstable search mode phases.

## 8 Experiments

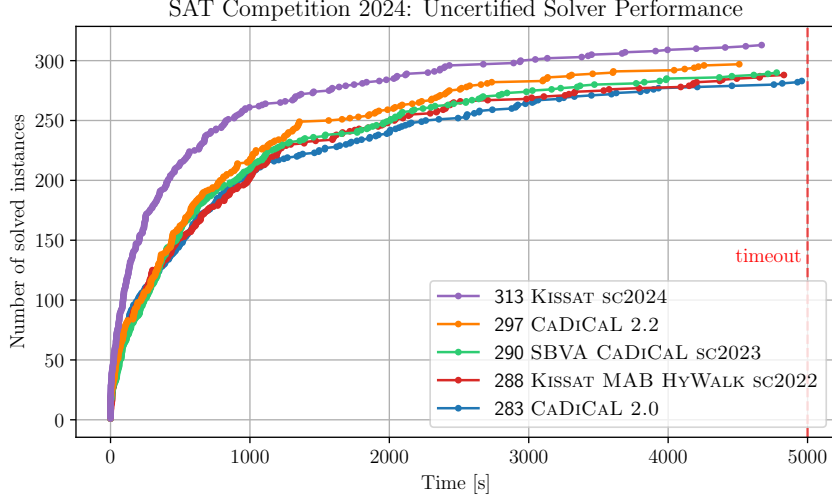
We used all 400 problems from the SAT Competition 2024 and 8-core Intel Xeon E5-2620 v4 CPUs running at 2.10 GHz with 127 GB RAM. All experimental data, source code, binaries, and scripts to produce plots are available on Zenodo [46]. The C++ source code increased from 46 kloc for CADICAL 2.0 [11] to 64 kloc for CADICAL 2.2 (22 kloc added and 3 kloc removed according to “git diff”).

In Fig. 2 we compare CADICAL 2.2 with the winners of 2022-2024, where it indeed performs better than CADICAL 2.0, but worse than KISSAT SC2024. SBVA-CADICAL SC2023 runs BVA preprocessing before running CADICAL 1.5.3, while KISSAT MAB HYWALK SC2022 has neither BVA nor congruence.

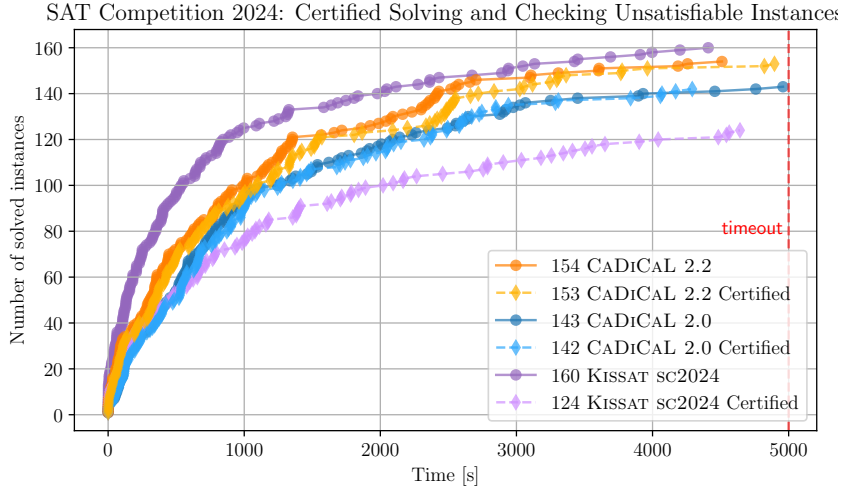
We further run our solvers CADICAL and KISSAT with proof production and proof checking with the proof checker LRAT-TRIM or DRAT-TRIM for all *unsatisfiable* instances (Fig. 3). CADICAL is the only solver supporting LRAT, while KISSAT can only produce DRAT proofs (simpler to generate, but way more expensive to check). In essence, LRAT proof checking is instant, while DRAT takes much longer – the slowest proofs take more than 7 hours due to BVA.

Finally, we evaluate robustness of our new approach using ticks, with respect to the alternation of unstable and stable mode (Fig. 4). With ticks mode alternation is more balanced, even though we can observe a slight imbalance towards unstable mode, especially as CADICAL starts in unstable mode. There are fewer extreme



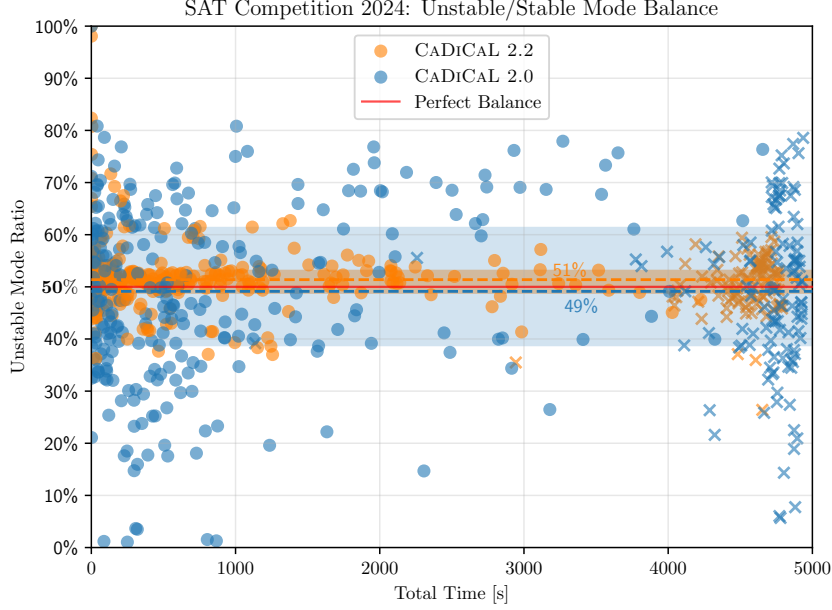


**Fig. 2.** We compare uncertified solving of the winners of the SAT Competitions from 2022 to 2024, i.e., KISSAT MAB HyWALK sc2022 [53], SBVA-CADiCAL sc2023 [31], and KISSAT sc2024 [12], against CADiCAL 2.0 [11] and the new CADiCAL 2.2 on the 400 benchmarks from the SAT Competition 2024 without producing proofs.



**Fig. 3.** We report the time for (1) solving without proof production (dots) and for (2) “Certified” solving (diamonds), which includes proof production and proof checking.

cases. The graph shows some runs hitting the timeout (crosses) with run-times of less than 3000s: for CADiCAL 2.2 this can be contributed to the local-search random-walk procedure which is not yet limited by ticks but by the number of flipped variables (flips). Limiting random walk with ticks is left to future work.



**Fig. 4.** Solving is split into search and simplification through pre- and inprocessing. Furthermore, searching is split into stable and unstable mode. An important goal of using ticks is to balance the time spent in these two modes. Thus we compare relative time spent in these modes for CADICAL 2.0 (using conflicts) and CADICAL 2.2 (ticks). Each dot corresponds to one of the 400 SAT Competition 2024 benchmarks. The  $x$ -coordinate gives the total time of the solver spent in both modes and the  $y$ -coordinate the fraction spent in unstable mode, i.e., on the 50% line the same time is spent in each mode. Unsolved instances are marked with a cross instead of a circle. We exclude 18 instances for CADICAL 2.0 and 28 instances for CADICAL 2.2, solved by initial preprocessing or the *lucky phase*, without any search. For each solver, CADICAL 2.0 in blue and CADICAL 2.2 in orange, we show the 25th, 50th (median) and 75th percentiles.

## 9 Conclusion

We successfully extended CADICAL with clausal congruence closure, clausal equivalence sweeping, and bounded variable addition, while continuing to support efficient linear proof production with hints. Scheduling based on ticks makes the solver more robust. Understanding the impact of these techniques on incremental solving and closing the gap between CADICAL and KISSAT on *satisfiable* instances (*cf.* Appendix) is left to future work. We also want to port binary backbone extraction [30] from KISSAT, the only technique not yet in CADICAL.

**Acknowledgements** This work was supported in part by the Austrian Science Fund (FWF) under project T-1306, W1255-N23, and S11408-N23, the state of Baden-Württemberg through bwHPC, the German Research Foundation (DFG) through grant INST 35/1597-1 FUGG, and by a gift from Intel Corporation.

## References

- [1] Audemard, G., Simon, L.: Predicting learnt clauses quality in modern SAT solvers. In: Boutilier, C. (ed.) IJCAI 2009, Proceedings of the 21st International Joint Conference on Artificial Intelligence, Pasadena, California, USA, July 11-17, 2009. pp. 399–404. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (2009), <http://ijcai.org/Proceedings/09/Papers/074.pdf>
- [2] Audemard, G., Simon, L.: On the Glucose SAT solver. *Int. J. Artif. Intell. Tools* **27**(1), 1840001:1–1840001:25 (2018). <https://doi.org/10.1142/S0218213018400018>
- [3] Baek, S., Carneiro, M., Heule, M.J.H.: A flexible proof format for SAT solver-elaborator communication. In: Groote, J.F., Larsen, K.G. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 27th International Conference, TACAS 2021, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2021, Luxembourg City, Luxembourg, March 27 - April 1, 2021, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 12651, pp. 59–75. Springer (2021). [https://doi.org/10.1007/978-3-030-72016-2\\_4](https://doi.org/10.1007/978-3-030-72016-2_4)
- [4] Baek, S., Carneiro, M., Heule, M.J.H.: A flexible proof format for SAT solver-elaborator communication. *Log. Methods Comput. Sci.* **18**(2) (2022). [https://doi.org/10.46298/LMCS-18\(2:3\)2022](https://doi.org/10.46298/LMCS-18(2:3)2022)
- [5] Balyo, T., Fröhlich, A., Heule, M., Biere, A.: Everything you always wanted to know about blocked sets (but were afraid to ask). In: Sinz, C., Egly, U. (eds.) *Theory and Applications of Satisfiability Testing - SAT 2014 - 17th International Conference, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 14-17, 2014. Proceedings. LNCS*, vol. 8561, pp. 317–332. Springer (2014). [https://doi.org/10.1007/978-3-319-09284-3\\_24](https://doi.org/10.1007/978-3-319-09284-3_24)
- [6] Balyo, T., Heule, M.J.H., Iser, M., Jarvisalo, M., Suda, M. (eds.): *Proceedings of SAT Competition 2023: Solver, Benchmark and Proof Checker Descriptions*. Department of Computer Science Series of Publications B, Department of Computer Science, University of Helsinki, Finland (2023)
- [7] Biere, A.: Lingeling, Plingeling, PicoSAT and Precosat at SAT Race 2010. Tech. Rep. 10/1, Johannes Kepler University Linz, FMV Reports Series, Institute for Formal Models and Verification, Johannes Kepler University, Altenbergerstr. 69, 4040 Linz, Austria (2010). <https://doi.org/10.350/fmvtr.2010-1>
- [8] Biere, A.: Lingeling essentials, a tutorial on design and implementation aspects of the the sat solver lingeling. In: Berre, D.L. (ed.) *POS-14. Fifth Pragmatics of SAT workshop. EPiC Series in Computing*, vol. 27, p. 88. EasyChair (2014). <https://doi.org/10.29007/jhd7>, [/publications/paper/xJs](https://publications/paper/xJs)
- [9] Biere, A.: Dimocheck dimacs model checker (Last access, February 2025), <https://github.com/arminbiere/dimocheck>, source code
- [10] Biere, A.: LRAT trimmer (Last access, March 2023), <https://github.com/arminbiere/lrat-trim>, source code
- [11] Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F.: Cadical 2.0. In: Gurfinkel, A., Ganesh, V. (eds.) *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I. LNCS*, vol. 14681, pp. 133–152. Springer (2024). [https://doi.org/10.1007/978-3-031-65627-9\\_7](https://doi.org/10.1007/978-3-031-65627-9_7)
- [12] Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F.: CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024. In: Heule, M., Iser, M., Jarvisalo, M., Suda, M. (eds.) *Proc. of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions*. Department of Computer Science Report Series B, vol. B-2024-1, pp. 8–10. University of Helsinki (2024)

- [13] Biere, A., Fazekas, K., Fleury, M., Froleyks, N.: Clausal congruence closure. In: Chakraborty, S., Jiang, J.R. (eds.) 27th International Conference on Theory and Applications of Satisfiability Testing, SAT 2024, August 21-24, 2024, Pune, India. LIPIcs, vol. 305, pp. 6:1–6:25. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2024). <https://doi.org/10.4230/LIPICS.SAT.2024.6>
- [14] Biere, A., Fazekas, K., Fleury, M., Froleyks, N.: Clausal Congruence Closure. In: 27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024). Leibniz International Proceedings in Informatics (LIPIcs), vol. 305, pp. 6:1–6:25 (2024). <https://doi.org/10.4230/LIPICS.SAT.2024.6>
- [15] Biere, A., Fazekas, K., Fleury, M., Froleyks, N.: Clausal equivalence sweeping. In: Narodytska, N., Rümmer, P. (eds.) Proceedings 24th International Conference on Formal Methods in Computer-Aided Design (FMCAD’24). pp. 236–241. TU Wien Academic Press (2024). [https://doi.org/10.34727/2024/isbn.978-3-85448-065-5\\_29](https://doi.org/10.34727/2024/isbn.978-3-85448-065-5_29)
- [16] Biere, A., Fazekas, K., Fleury, M., Heisinger, M.: CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020. In: Balyo, T., Froleyks, N., Heule, M.J.H., Iser, M., Jarvisalo, M., Suda, M. (eds.) Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions. Department of Computer Science Report Series B, vol. B-2020-1, pp. 51–53. University of Helsinki (2020)
- [17] Biere, A., Froleyks, N., Wang, W.: CadiBack: Extracting backbones with CaDiCaL. In: Mahajan, M., Slivovsky, F. (eds.) 26th International Conference on Theory and Applications of Satisfiability Testing, SAT 2023, July 4-8, 2023, Alghero, Italy. LIPIcs, vol. 271, pp. 3:1–3:12. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2023). <https://doi.org/10.4230/LIPICS.SAT.2023.3>
- [18] Biere, A., Jarvisalo, M., Kiesl, B.: Preprocessing in SAT solving. In: Biere, A., Heule, M.J.H., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications, vol. 336, pp. 391 – 435. IOS Press, 2nd edition edn. (2021)
- [19] Bogaerts, B., Gocht, S., McCreesh, C., Nordström, J.: Certified dominance and symmetry breaking for combinatorial optimisation. Journal of Artificial Intelligence Research **77**, 1539–1589 (Aug 2023), preliminary version in *AAAI ’22*
- [20] Cruz-Filipe, L., Heule, M.J.H., Hunt, Warren A., J., Kaufmann, M., Schneider-Kamp, P.: Efficient certified RAT verification. In: de Moura, L. (ed.) Automated Deduction - CADE 26 - 26th International Conference on Automated Deduction, Gothenburg, Sweden, August 6-11, 2017, Proceedings. LNCS, vol. 10395, pp. 220–236. Springer (2017). [https://doi.org/10.1007/978-3-319-63046-5\\_14](https://doi.org/10.1007/978-3-319-63046-5_14)
- [21] Cruz-Filipe, L., Heule, M.J.H., Hunt, W.A., Kaufmann, M., Schneider-Kamp, P.: Efficient certified RAT verification. In: de Moura, L. (ed.) Automated Deduction – CADE 26. pp. 220–236. Springer International Publishing, Cham (2017)
- [22] Eén, N., Sörensson, N.: An extensible SAT-solver. In: Giunchiglia, E., Tacchella, A. (eds.) Theory and Applications of Satisfiability Testing, 6th International Conference, SAT 2003. Santa Margherita Ligure, Italy, May 5-8, 2003 Selected Revised Papers. LNCS, vol. 2919, pp. 502–518. Springer (2003). [https://doi.org/10.1007/978-3-540-24605-3\\_37](https://doi.org/10.1007/978-3-540-24605-3_37)
- [23] Fazekas, K.: On SAT-based Solution Methods for Computational Problems. Ph.D. thesis, Informatik, Johannes Kepler University Linz (2020)
- [24] Fazekas, K., Biere, A., Scholl, C.: Incremental inprocessing in SAT solving. In: Janota, M., Lynce, I. (eds.) Theory and Applications of Satisfiability Testing - SAT 2019 - 22nd International Conference, SAT 2019, Lisbon, Portugal, July 9-12, 2019, Proceedings. LNCS, vol. 11628, pp. 136–154. Springer (2019). [https://doi.org/10.1007/978-3-030-24258-9\\_9](https://doi.org/10.1007/978-3-030-24258-9_9)

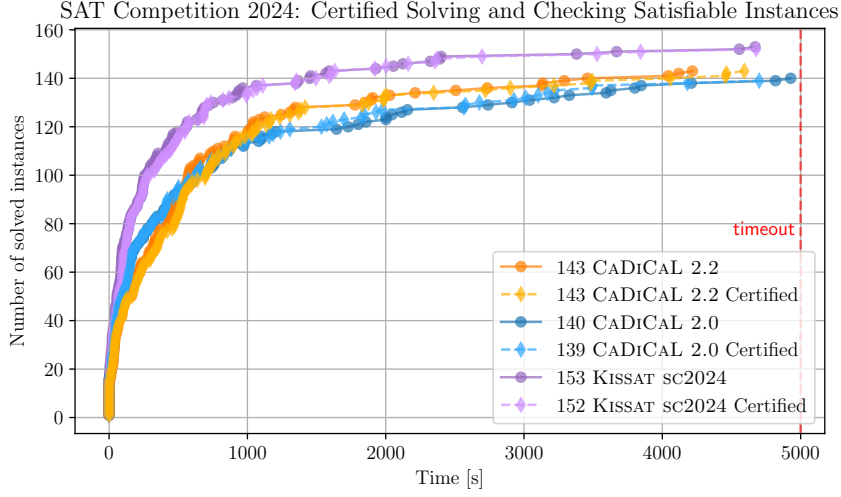
- [25] Fazekas, K., Niemetz, A., Preiner, M., Kirchweger, M., Szeider, S., Biere, A.: IPASIR-UP: user propagators for CDCL. In: Mahajan, M., Slivovsky, F. (eds.) 26th International Conference on Theory and Applications of Satisfiability Testing, SAT 2023, July 4-8, 2023, Alghero, Italy. LIPIcs, vol. 271, pp. 8:1–8:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2023). <https://doi.org/10.4230/LIPICS.SAT.2023.8>
- [26] Fazekas, K., Niemetz, A., Preiner, M., Kirchweger, M., Szeider, S., Biere, A.: Satisfiability modulo user propagators. *J. Artif. Intell. Res.* **81**, 989–1017 (2024). <https://doi.org/10.1613/JAIR.1.16163>
- [27] Fazekas, K., Pollitt, F., Fleury, M., Biere, A.: Certifying incremental SAT solving. In: Bjørner, N.S., Heule, M., Voronkov, A. (eds.) LPAR 2024: Proceedings of 25th Conference on Logic for Programming, Artificial Intelligence and Reasoning, Port Louis, Mauritius, May 26-31, 2024. EPIc Series in Computing, vol. 100, pp. 321–340. EasyChair (2024). <https://doi.org/10.29007/PDCC>
- [28] Fazekas, K., Pollitt, F., Fleury, M., Biere, A.: Incremental proofs for bounded model checking. In: Kunz, W. (ed.) 27th GMM/ITG/GI Workshop on Methods and Description Languages for Modelling and Verification of Circuits and Systems (MBMV’24), Kaiserslautern, Germany. ITG-Fachberichte, vol. 314, pp. 133–143. VDE Verlag (2024)
- [29] Fleury, M., Biere, A.: Mining definitions in Kissat with Kittens. *Formal Methods Syst. Des.* **60**(3), 381–404 (2022). <https://doi.org/10.1007/S10703-023-00421-2>
- [30] Froleys, N., Yu, E., Biere, A.: BIG backbones. In: Nadel, A., Rozier, K.Y. (eds.) Formal Methods in Computer-Aided Design, FMCAD 2023, Ames, IA, USA, October 24-27, 2023. pp. 162–167. IEEE (2023). [https://doi.org/10.34727/2023/ISBN.978-3-85448-060-0\\_24](https://doi.org/10.34727/2023/ISBN.978-3-85448-060-0_24)
- [31] Haberlandt, A., Green, H.: SBVA-CaDiCaL and SBVA-Kissat: Structured bounded variable addition. In: Balyo, T., Froleys, N., Heule, M.J.H., Iser, M., Järvisalo, M., Suda, M. (eds.) Proc. of SAT Competition 2023 – Solver and Benchmark Descriptions. Department of Computer Science Report Series B, vol. B-2023-1, p. 18. University of Helsinki (2023)
- [32] Heule, M.J.H.: Proofs of unsatisfiability. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability - Second Edition, Frontiers in Artificial Intelligence and Applications, vol. 336, pp. 635–668. IOS Press (2021). <https://doi.org/10.3233/FAIA200998>
- [33] Heule, M.J.H., Biere, A.: Blocked clause decomposition. In: McMillan, K.L., Middeldorp, A., Voronkov, A. (eds.) Logic for Programming, Artificial Intelligence, and Reasoning - 19th International Conference, LPAR-19, Stellenbosch, South Africa, December 14-19, 2013. Proceedings. LNCS, vol. 8312, pp. 423–438. Springer (2013). [https://doi.org/10.1007/978-3-642-45221-5\\_29](https://doi.org/10.1007/978-3-642-45221-5_29)
- [34] Heule, M.J.H., Biere, A.: Proofs for satisfiability problems. In: All about Proofs, Proofs for All (APPA), Math. Logic and Foundations, vol. 55. College Pub. (2015)
- [35] Heule, M.J.H., Kiesl, B., Biere, A.: Strong extension-free proof systems. *J. Autom. Reason.* **64**(3), 533–554 (2020). <https://doi.org/10.1007/S10817-019-09516-0>, <https://doi.org/10.1007/s10817-019-09516-0>
- [36] Heule, M.J., Hunt, W.A., Wetzler, N.: Trimming while checking clausal proofs. In: 2013 Formal Methods in Computer-Aided Design. pp. 181–188 (2013). <https://doi.org/10.1109/FMCAD.2013.6679408>
- [37] Ignatiev, A., Morgado, A., Marques-Silva, J.: PySAT: A Python toolkit for prototyping with SAT oracles. In: Beyersdorff, O., Wintersteiger, C.M. (eds.) Theory and Applications of Satisfiability Testing - SAT 2018 - 21st International Conference,

- SAT 2018, Held as Part of the Federated Logic Conference, FloC 2018, Oxford, UK, July 9-12, 2018, Proceedings. LNCS, vol. 10929, pp. 428–437. Springer (2018). [https://doi.org/10.1007/978-3-319-94144-8\\_26](https://doi.org/10.1007/978-3-319-94144-8_26)
- [38] Ignatiev, A., Tan, Z.L., Karamanos, C.: Towards universally accessible SAT technology. In: SAT. pp. 4:1–4:11 (2024). <https://doi.org/10.4230/LIPICS.SAT.2024.16>, <https://doi.org/10.4230/LIPICS.SAT.2024.16>
- [39] Knuth, D.E.: The Art of Computer Programming, Volume 4, Fascicle 4: Generating All Trees—History of Combinatorial Generation (Art of Computer Programming). Addison-Wesley Professional (2006)
- [40] Lammich, P.: The GRAT tool chain - efficient (UN)SAT certificate checking with formal correctness guarantees. In: Gaspers, S., Walsh, T. (eds.) Theory and Applications of Satisfiability Testing - SAT 2017 - 20th International Conference, Melbourne, VIC, Australia, August 28 - September 1, 2017, Proceedings. LNCS, vol. 10491, pp. 457–463. Springer (2017). [https://doi.org/10.1007/978-3-319-66263-3\\_29](https://doi.org/10.1007/978-3-319-66263-3_29), [https://doi.org/10.1007/978-3-319-66263-3\\_29](https://doi.org/10.1007/978-3-319-66263-3_29)
- [41] Le Berre, D., Roussel, O., Simon, L.: SAT competition 2009: Benchmark submission guidelines. <https://web.archive.org/web/20190325181937/https://www.satcompetition.org/2009/format-benchmarks2009.html>, accessed: 2024-01-15
- [42] Manthey, N., Heule, M.J.H., Biere, A.: Automated reencoding of boolean formulas. In: Biere, A., Nahir, A., Vos, T. (eds.) Hardware and Software: Verification and Testing. pp. 102–117. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
- [43] Nabeshima, H., Fukiage, T., Obitsu, Y., Lu, X.N., Inoue, K.: Dps: a framework for deterministic parallel sat solvers. EasyChair Preprint 8553 (EasyChair, 2022)
- [44] Oh, C.: Between SAT and UNSAT: the fundamental difference in CDCL SAT. In: Theory and Applications of Satisfiability Testing—SAT 2015—18th International Conference, Austin, TX, USA, September 24-27, 2015, Proceedings. pp. 307–323 (2015). [https://doi.org/10.1007/978-3-319-24318-4\\_23](https://doi.org/10.1007/978-3-319-24318-4_23)
- [45] Pollitt, F., Fleury, M., Biere, A.: Faster LRAT checking than solving with cadical. In: Mahajan, M., Slivovsky, F. (eds.) 26th International Conference on Theory and Applications of Satisfiability Testing, SAT 2023, July 4-8, 2023, Alghero, Italy. LIPIcs, vol. 271, pp. 21:1–21:12. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2023). <https://doi.org/10.4230/LIPICS.SAT.2023.21>
- [46] Pollitt, F., Fleury, M., Fazekas, K., Froleyks, N., Biere, A.: CaDiCaL 2.2 system description artifact (Feb 2025). <https://doi.org/10.5281/zenodo.14944298>
- [47] Soos, M., Gocht, S., Meel, K.S.: Tinted, detached, and lazy CNF-XOR solving and its applications to counting and sampling. In: Lahiri, S.K., Wang, C. (eds.) Computer Aided Verification - 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21-24, 2020, Proceedings, Part I. LNCS, vol. 12224, pp. 463–484. Springer (2020). [https://doi.org/10.1007/978-3-030-53288-8\\_22](https://doi.org/10.1007/978-3-030-53288-8_22)
- [48] Soos, M., Meel, K.S.: BIRD: engineering an efficient CNF-XOR SAT solver and its applications to approximate model counting. In: The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019. pp. 1592–1599. AAAI Press (2019). <https://doi.org/10.1609/aaai.v33i01.33011592>
- [49] Soos, M., Nohl, K., Castelluccia, C.: Extending SAT solvers to cryptographic problems. In: Kullmann, O. (ed.) Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009. Proceedings. LNCS, vol. 5584, pp. 244–257. Springer (2009). [https://doi.org/10.1007/978-3-642-02777-2\\_24](https://doi.org/10.1007/978-3-642-02777-2_24)

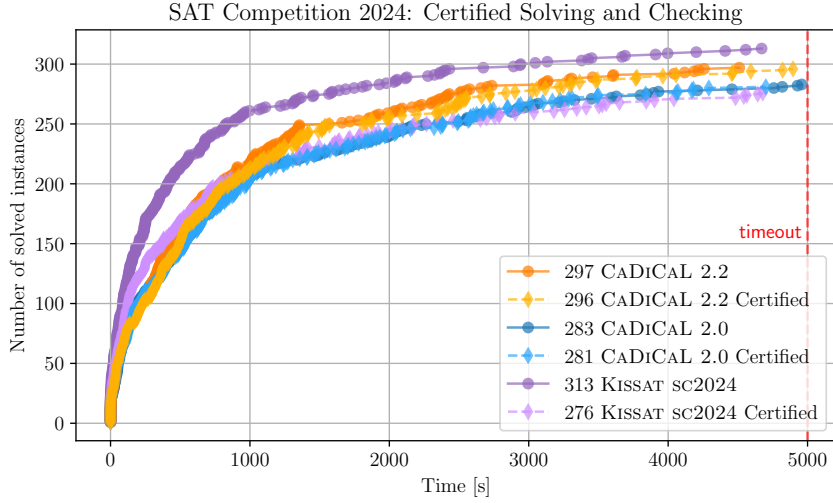
- [50] Tan, Y.K., Yang, J., Soos, M., Myreen, M.O., Meel, K.S.: Formally certified approximate model counting. In: Gurfinkel, A., Ganesh, V. (eds.) *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I*. LNCS, vol. 14681, pp. 153–177. Springer (2024). [https://doi.org/10.1007/978-3-031-65627-9\\_8](https://doi.org/10.1007/978-3-031-65627-9_8)
- [51] Wetzler, N., Heule, M.J.H., Hunt, Warren A., J.: DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In: Sinz, C., Egly, U. (eds.) *Theory and Applications of Satisfiability Testing - SAT 2014 - 17th International Conference, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 14-17, 2014. Proceedings*. LNCS, vol. 8561, pp. 422–429. Springer (2014). [https://doi.org/10.1007/978-3-319-09284-3\\_31](https://doi.org/10.1007/978-3-319-09284-3_31)
- [52] Yamada, A., Biere, A., Artho, C., Kitamura, T., Choi, E.: Greedy combinatorial test case generation using unsatisfiable cores. In: Lo, D., Apel, S., Khurshid, S. (eds.) *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, ASE 2016, Singapore, September 3-7, 2016*. pp. 614–624. ACM (2016). <https://doi.org/10.1145/2970276.2970335>
- [53] Zheng, J., He, K., Chen, Z., Zhou, J., Li, C.M.: Combining hybrid walking strategy with Kissat MAB, CaDiCaL, and LStech-Maple. In: Balyo, T., Heule, M., Iser, M., Järvisalo, M., Suda, M. (eds.) *Proceedings of SAT Competition 2022: Solver and Benchmark Descriptions*. Department of Computer Science Series of Publications B, Department of Computer Science, University of Helsinki, Finland (2022)



## A Appendix



**Fig. 5.** Similar to Fig. 3 focusing on satisfiable instances. This includes both solving and checking time of the produced model using our new model checker DiMoCHECK [9].



**Fig. 6.** As Fig. 3 but with both unsatisfiable and satisfiable benchmarks. This includes proof production, proof checking for unsatisfiable resp. model validation for satisfiable solver results. For certified solving we use our model checker DiMoCHECK [9] for satisfiable instances and our new version of LRAT-TRIM [10] for unsatisfiable benchmarks.



```

#include "cadical.hpp"
#include <cassert>
#include <iostream>

CaDiCaL::Solver *solver = new CaDiCaL::Solver ();

int main () {
    enum { TIE = 1, SHIRT = 2, HAT = 3 };

    solver->set ("binary", 0);
    solver->set ("lidrup", 1);
    solver->trace_proof ("propagate.lidrup");
    solver->set ("flushproof", 1);

    solver->add (-TIE), solver->add (SHIRT), solver->add (0);
    solver->add (TIE), solver->add (SHIRT), solver->add (0);
    solver->add (-TIE), solver->add (-SHIRT), solver->add (0);
    solver->add (TIE), solver->add (SHIRT), solver->add (HAT), solver->add (0);

    solver->assume (-TIE);
    int res = solver->propagate ();
    assert (res == 0);

    std::vector<int> implicants;
    solver->implied (implicants);
    assert(implicants.size() > 0 && implicants.size() < 3);

    solver->assume (TIE);
    res = solver->propagate ();
    assert (res == 20);
    assert(solver->failed(TIE));

    solver->assume (SHIRT);
    solver->assume (HAT);
    res = solver->propagate ();
    assert (res == 10);
    assert (solver->val(TIE) == -1);

    return 0;
}

```

```

i 1 -1 2 0
i 2 1 2 0
i 3 -1 -2 0
i 4 1 2 3 0
q -1 0
s UNKNOWN
e -1 2 0
q 1 0
l 5 -1 0 1 3 0
l 6 2 0 5 2 0
s UNSATISFIABLE
u 1 0 5 0
q 2 3 0
s SATISFIABLE
m -1 2 3 0

```

**Fig. 7.** An example of using new API features and its produced proof.